

ΒΑΣΙΛΕΙΟΣ ΚΟΚΚΙΝΟΓΕΝΗΣ IT2021042

Κείμενο Τεκμηρίωσης για την Εργασία στο IR & NLP

Μηχανή Αναζήτησης πάνω σε Wikipedia Movie Plots

1. Εισαγωγή

Στην παρούσα εργασία δημιουργήθηκε μια μηχανή αναζήτησης για πλοκές ταινιών από την Wikipedia. Ο στόχος της είναι η αποτελεσματική ανάκτηση και κατάταξη των πιο συναφών πλοκών ταινιών, σύμφωνα με την αναζήτηση που πραγματοποιεί ο χρήστης. Η μηχανή υποστηρίζει δύο τύπους αναζήτησης: βασισμένη σε λέξεις-κλειδιά (BM25) και σημασιολογική αναζήτηση (SBERT με FAISS).

Το dataset που χρησιμοποιήθηκε βρίσκεται εδώ:

<https://www.kaggle.com/datasets/jrobischon/wikipedia-movie-plots>

2. Αρχιτεκτονική και Τεχνολογίες

Για την υλοποίηση της μηχανής αναζήτησης χρησιμοποιήθηκαν οι εξής τεχνολογίες και εργαλεία:

- **Python:** Γλώσσα προγραμματισμού για την ανάπτυξη της μηχανής αναζήτησης.
- **Flask:** Framework για την ανάπτυξη του server.
- **FAISS:** Βιβλιοθήκη για την αποτελεσματική αναζήτηση και ευρετηρίαση σημασιολογικών αναπαραστάσεων.
- **SBERT:** Μοντέλο για τη δημιουργία σημασιολογικών ενσωματώσεων.
- **BM25:** Αλγόριθμος για αναζήτηση βασισμένη σε λέξεις-κλειδιά.
- **HTML & JavaScript:** Για την ανάπτυξη της διεπαφής χρήστη.

3. Διαδικασία Ανάπτυξης

3.1 Φάση Φόρτωσης και Επεξεργασίας Δεδομένων

- **Φόρτωση Δεδομένων:** Το σύνολο δεδομένων περιλαμβάνει πλοκές ταινιών, τις οποίες φόρτωσα χρησιμοποιώντας την βιβλιοθήκη **pandas**.
- **Δημιουργία ID Εγγράφων:** Κάθε ταινία αντιστοιχίζεται με ένα μοναδικό **doc_id**.

- **Καθαρισμός Δεδομένων:** Αντικατέστησα τις τιμές "unknown" με **NaN** και αφαίρεσα τα διπλότυπα.
- **Επεξεργασία Γραμμών:** Εξήγαγε σημαντικά πεδία, όπως τον τίτλο και την πλοκή, και δημιούργησα έναν συνδυασμένο αξιολογητή για την αναζήτηση.

3.2 Δημιουργία Ενσωματώσεων (Embeddings)

- **SBERT:** Χρησιμοποίησα το μοντέλο **all-MiniLM-L6-v2** για να κωδικοποιήσω τα κείμενα σε διανύσματα.
- **BM25:** Εφάρμοσα tokenization για τα δεδομένα και υπολόγισα την IDF για την κατάταξη των εγγράφων με βάση τη συχνότητα των λέξεων.

3.3 Δημιουργία Ευρετηρίου (Indexing)

- **SBERT με FAISS:** Δημιούργησα ένα ευρετήριο τύπου **FlatIP** με το FAISS για την αναζήτηση με cosine similarity, και κανονικοποίησα τις ενσωματώσεις για την καλύτερη σύγκριση.

3.4 Ανάκτηση και Κατάταξη Αποτελεσμάτων

- **SBERT Embeddings με FAISS:** Ανακτώ τις ενσωματώσεις των εγγράφων και εκτελώ αναζητήσεις χρησιμοποιώντας το FAISS για γρήγορη και αποτελεσματική κατάταξη.
- **BM25:** Υπολογίζω την αναλογία TF-IDF για κάθε έγγραφο και ταξινομώ τα αποτελέσματα σύμφωνα με τη συνάφεια.

3.5 Αξιολόγηση

- **Golden Answers:** Χρησιμοποίησα ένα γλωσσικό μοντέλο (LLM) για την αξιολόγηση της συνάφειας των αποτελεσμάτων. Το μοντέλο κρίνει αν κάθε έγγραφο είναι σχετικό ή όχι και επιστρέφει τα αποτελέσματα με τις αντίστοιχες ετικέτες συνάφειας.
-

4. Προαπαιτούμενα για την Υλοποίηση των Χρυσών Απαντήσεων (Golden Answers LLM)

Για να επιτευχθεί η αυτόματη αξιολόγηση της συνάφειας χρησιμοποιώντας το **llm μοντέλο** από το Hugging Face, ακολουθήστε τα παρακάτω βήματα:

4.1 Δημιουργία Λογαριασμού στο Hugging Face

- Επισκεφτείτε το [Hugging Face](https://huggingface.co) και δημιουργήστε έναν λογαριασμό, αν δεν έχετε ήδη.

4.2 Ανάκτηση του API Key σας

- Μετά την είσοδό σας στον λογαριασμό σας, μεταβείτε στην ενότητα Access Tokens των ρυθμίσεων του λογαριασμού σας.
- Δημιουργήστε ένα νέο API key ακολουθώντας τις οδηγίες της σελίδας.

4.3 Ενεργοποίηση Δικαιωμάτων Inference

- Βεβαιωθείτε ότι το API key που δημιουργήσατε έχει ενεργοποιημένα τα **Inference permissions**. Αυτή η άδεια είναι απαραίτητη για να αλληλεπιδράσετε με μοντέλα όπως το Mistral-7B για την εκτέλεση εργασιών inference.

4.4 Δημιουργία Αρχείου `.env`

- Στον ριζικό φάκελο του έργου σας, δημιουργήστε ένα αρχείο με όνομα `.env`.
- Προσθέστε την παρακάτω γραμμή στο αρχείο, αντικαθιστώντας το `{api_key}` με το πραγματικό σας API key:

```
HF_API_KEY={api_key}
```

4.5 Χρησιμοποιούμενο Μοντέλο

- Το μοντέλο που χρησιμοποιήσατε για την αξιολόγηση συνάφειας είναι το **Mistral-7B-Instruct**, το οποίο μπορεί να προσπελαστεί μέσω του Hugging Face API:

<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>

Αυτό το μοντέλο χρησιμοποιείται για την αυτόματη αξιολόγηση της συνάφειας των εγγράφων με βάση το ερώτημα και τα κορυφαία αποτελέσματα που επιστρέφονται.

5. Routes του Flask Server + Html Interface

5.1 Δημιουργία Διεπαφής Χρήστη (Frontend)

Η διεπαφή χρήστη αναπτύχθηκε με HTML και JavaScript, επιτρέποντας στους χρήστες να εισάγουν το query τους, να επιλέγουν το επιθυμητό πεδίο μεταδεδομένων και τη μέθοδο αναζήτησης.

5.2 /search:

Η διαδρομή αυτή επεξεργάζεται το query σε πραγματικό χρόνο και επιστρέφει τα top 10 αποτελέσματα με βάση την επιλεγμένη μέθοδο αναζήτησης και το πεδίο μεταδεδομένων (τίτλος, πλοκή, κλπ.). Υποστηρίζει αναζητήσεις βασισμένες σε BM25 ή σημασιολογική αναζήτηση με FAISS.

5.3 /judge:

Η διαδρομή αυτή επιτρέπει την αυτόματη αξιολόγηση των αποτελεσμάτων. Αποστέλλει το query και τα top 10 αποτελέσματα σε ένα γλωσσικό μοντέλο (LLM) και επιστρέφει την κρίση του μοντέλου για το αν τα αποτελέσματα είναι σχετικά ή όχι.

5.4 Εκκίνηση του server και του αρχείου main

Για να ξεκινήσετε τον server Flask, χρησιμοποιήστε την παρακάτω εντολή:

```
python server.py
```

Σημείωση: Το αρχείο `main.py` θα λειτουργήσει μόνο αν ο server βρίσκεται ήδη σε λειτουργία. Δεν μπορεί να εκτελεστεί από μόνο του.

6. Αξιολόγηση Απόδοσης

Για την αξιολόγηση της απόδοσης της μηχανής αναζήτησης, χρησιμοποιήθηκε η ακρίβεια των πρώτων 10 αποτελεσμάτων σε σχέση με τα golden answers. Παρατηρήθηκε ότι η χρήση του MiniLM με FAISS παρείχε τα καλύτερα αποτελέσματα σε ακρίβεια και ταχύτητα, ενώ το BM25 έδειξε επίσης ανταγωνιστική απόδοση.

7. Συμπεράσματα και Περαιτέρω Βελτιώσεις

Η μηχανή αναζήτησης προσφέρει γρήγορη και ακριβή ανάκτηση αποτελεσμάτων. Περαιτέρω βελτιώσεις περιλαμβάνουν την εξερεύνηση άλλων μοντέλων ενσωματώσεων, όπως το GloVe ή το Doc2Vec, για την αξιολόγηση της απόδοσης σε διαφορετικές συνθήκες αναζήτησης.